

# TOAE209/TOAH209 – Design and Analysis of Algorithms

## Week 6: Theoretical Questions

Foreign Trade University

### Instructions

Part A contains 17 questions requiring written explanations, short proofs, or recurrence-solving. Part B is a 10-question quiz (true/false and multiple choice). Attempt every question on your own before consulting Part C (Solutions) at the end of this document.

### Part A: Theory Questions

- A1.** Describe the *divide-and-conquer* paradigm in general terms. Name and explain its three steps, and write the general form of the running-time recurrence that results.
- A2.** State the MERGE subroutine for two sorted lists and prove that it produces a sorted list containing exactly the elements of both inputs. State a loop invariant and use it in your proof.
- A3.** Prove that MERGESORT correctly sorts any input array, by strong induction on the array length.
- A4.** Write down the recurrence for mergesort's running time and solve it by *unrolling* (repeated substitution). Show the intermediate steps and identify the number of levels.
- A5.** Solve the same mergesort recurrence  $T(n) = 2T(n/2) + cn$  using a *recursion tree*. For each level  $k$ , give the number of nodes, the size of each node, and the total work at that level; then sum over all levels.
- A6.** State the **Master Theorem** in full, including all three cases and the condition (comparison of  $d$  with  $\log_b a$ ) that selects each case.
- A7.** For each of the following recurrences, apply the Master Theorem and give the  $\Theta$ -class: (a)  $T(n) = 2T(n/2) + \Theta(n)$ ; (b)  $T(n) = T(n/2) + \Theta(1)$ ; (c)  $T(n) = 3T(n/2) + \Theta(n)$ ; (d)  $T(n) = 4T(n/2) + \Theta(n)$ ; (e)  $T(n) = 2T(n/2) + \Theta(n^2)$ . State which case applies and why.
- A8.** For each recurrence, give the  $\Theta$ -class via the Master Theorem and identify the case: (a)  $T(n) = 9T(n/3) + \Theta(n)$ ; (b)  $T(n) = T(n/2) + \Theta(n)$ ; (c)  $T(n) = 8T(n/2) + \Theta(n^3)$ ; (d)  $T(n) = 7T(n/2) + \Theta(n^2)$ .
- A9.** Define an *inversion* of an array. How many inversions does a sorted array of length  $n$  have? How many does a strictly reverse-sorted array of length  $n$  have? Justify the reverse-sorted count.

- A10.** Explain how counting inversions can be done in  $O(n \log n)$  by modifying mergesort. In particular, when merging sorted halves  $L$  and  $R$  and the algorithm emits an element of  $R$  before exhausting  $L$ , exactly how many inversions does that single step account for, and why?
- A11.** Write down and justify the recurrence for the inversion-counting algorithm. Why is it the same order as mergesort itself?
- A12.** State the recurrence for binary search and solve it. Then explain why binary search is "divide and conquer with  $a = 1$ " and what that means for the shape of its recursion tree (compared to mergesort's).
- A13.** Describe the divide-and-conquer algorithm for the maximum-subarray problem. In particular, describe the three cases the maximum subarray can fall into, and how the *cross-boundary* case is computed in  $O(n)$ . State and solve the recurrence.
- A14.** Explain exponentiation by squaring. Write the recursive identity for  $x^n$  (even and odd cases), state the recurrence for the number of multiplications, solve it, and explain why computing  $x^{n/2}$  *once* (rather than twice) is essential to achieving  $\Theta(\log n)$ .
- A15.** Describe *quickselect* for finding the  $k$ -th smallest element. Explain why, with a random pivot, the *expected* running time is  $\Theta(n)$ , and why the Master Theorem does not directly apply to its recurrence. What is the worst-case running time, and when does it occur?
- A16.** Explain the divide-and-conquer algorithm for finding a *majority element* (a value occurring more than  $n/2$  times). Why does it suffice to consider only the majority candidates of the two halves in the combine step? State and solve the recurrence.
- A17.** Karatsuba multiplication satisfies  $T(n) = 3T(n/2) + \Theta(n)$ , while the naive divide-and-conquer multiplication satisfies  $T(n) = 4T(n/2) + \Theta(n)$ . Solve both recurrences with the Master Theorem and explain precisely why reducing the number of recursive multiplications from 4 to 3 changes the asymptotic running time (and why shaving the  $\Theta(n)$  combine term would *not* help).

## Part B: Quiz

**B1. (Multiple choice)** The recurrence for mergesort is:

- (a)  $T(n) = T(n-1) + \Theta(n)$
- (b)  $T(n) = 2T(n/2) + \Theta(n)$
- (c)  $T(n) = 2T(n/2) + \Theta(1)$
- (d)  $T(n) = T(n/2) + \Theta(n)$

**B2. (True/False)** Merging two sorted lists of total length  $n$  can be done in  $O(n)$  time.

**B3. (Multiple choice)** By the Master Theorem,  $T(n) = 2T(n/2) + \Theta(n)$  solves to:

- (a)  $\Theta(n)$
- (b)  $\Theta(n \log n)$
- (c)  $\Theta(n^2)$
- (d)  $\Theta(\log n)$

**B4. (Multiple choice)** For  $T(n) = aT(n/b) + \Theta(n^d)$ , Case 1 of the Master Theorem (where  $T(n) = \Theta(n^{\log_b a})$ ) applies when:

- (a)  $d > \log_b a$
- (b)  $d = \log_b a$
- (c)  $d < \log_b a$
- (d)  $a = b$

**B5. (Short answer)** How many inversions does the array  $[3, 2, 1]$  have? List them.

**B6. (True/False)** Counting the inversions of an  $n$ -element array requires  $\Theta(n^2)$  time in the worst case; no faster algorithm is possible.

**B7. (Multiple choice)** The number of multiplications used by exponentiation by squaring to compute  $x^n$  is:

- (a)  $\Theta(n)$
- (b)  $\Theta(\log n)$
- (c)  $\Theta(n \log n)$
- (d)  $\Theta(\sqrt{n})$

**B8. (Multiple choice)** The *expected* running time of quickselect with a random pivot is:

- (a)  $\Theta(\log n)$
- (b)  $\Theta(n)$
- (c)  $\Theta(n \log n)$
- (d)  $\Theta(n^2)$

**B9. (True/False)** The divide-and-conquer maximum-subarray algorithm runs in  $\Theta(n \log n)$ , while Kadane's algorithm solves the same problem in  $\Theta(n)$ .

**B10. (Short answer)** Karatsuba's recurrence is  $T(n) = 3T(n/2) + \Theta(n)$ . Give its  $\Theta$ -class (a numeric exponent to three decimal places is fine).

## Part C: Solutions

### Part A

**A1. Divide and conquer** solves a problem by reducing it to smaller instances of the same problem. Its three steps are: **Divide** – split the size- $n$  input into  $a$  subproblems each of size about  $n/b$ ; **Conquer** – solve each subproblem recursively (stopping at small base cases); **Combine** – merge the subproblem solutions into a solution for the original instance, doing some additional work. The running time satisfies a recurrence

$$T(n) = aT(n/b) + (\text{divide} + \text{combine cost}),$$

typically written  $T(n) = aT(n/b) + \Theta(n^d)$ .

**A2.**  $\text{MERGE}(L, R)$  keeps pointers  $i, j$  into the two sorted lists and repeatedly appends  $\min(L[i], R[j])$  to the output  $M$ , advancing the corresponding pointer; when one list is exhausted, it appends the remaining tail of the other. *Loop invariant:* at the start of each iteration, (i)  $M$  is sorted, and (ii) every element already in  $M$  is  $\leq$  every element remaining in  $L[i:]$  and  $R[j:]$ . *Proof:* initially  $M$  is empty, so the invariant holds vacuously. Each iteration appends the smaller of  $L[i], R[j]$ , which (because  $L, R$  are sorted) is the smallest of all remaining elements, hence  $\leq$  everything still remaining and (by the invariant)  $\geq$  everything already in  $M$ ; so  $M$  stays sorted and the invariant is preserved. When the loop ends, the remaining tail of one list is internally sorted and all its elements are  $\geq$  everything in  $M$ , so appending it leaves  $M$  sorted. Each input element is appended exactly once, so  $M$  contains exactly the multiset union.

**A3.** *Strong induction on  $n = |A|$ .* **Base case:**  $n \leq 1$ ; a list of length 0 or 1 is already sorted and returned unchanged. **Inductive step:** for  $n \geq 2$ , both halves have length  $< n$ , so by the inductive hypothesis the recursive calls return them sorted. By A2,  $\text{MERGE}$  of two sorted halves returns a sorted permutation of their union, which is  $A$ . Hence  $\text{MERGESORT}(A)$  is correct.

**A4.** Unrolling  $T(n) = 2T(n/2) + cn$ :

$$\begin{aligned} T(n) &= 2T(n/2) + cn = 2(2T(n/4) + c\frac{n}{2}) + cn = 4T(n/4) + 2cn \\ &= 8T(n/8) + 3cn = \dots = 2^k T(n/2^k) + kcn. \end{aligned}$$

The recursion stops when  $n/2^k = 1$ , i.e.  $k = \log_2 n$ , giving  $2^k T(1) = n \cdot O(1) = O(n)$  plus accumulated combine cost  $kcn = cn \log_2 n$ . Therefore  $T(n) = \Theta(n \log n)$ . There are  $\log_2 n + 1$  levels.

**A5.** Recursion tree for  $T(n) = 2T(n/2) + cn$ : level  $k$  (root is level 0) has  $2^k$  nodes, each of size  $n/2^k$ , each doing  $c(n/2^k)$  combine work. Total work at level  $k$  is  $2^k \cdot c(n/2^k) = cn$  – the same at every level. There are  $\log_2 n + 1$  levels, so the total is  $cn(\log_2 n + 1) = \Theta(n \log n)$ .

**A6. Master Theorem.** For  $T(n) = aT(n/b) + \Theta(n^d)$  with  $a \geq 1$ ,  $b > 1$ ,  $d \geq 0$ :

- **Case 1** ( $d < \log_b a$ , i.e.  $a > b^d$ ):  $T(n) = \Theta(n^{\log_b a})$  – leaves dominate.
- **Case 2** ( $d = \log_b a$ , i.e.  $a = b^d$ ):  $T(n) = \Theta(n^d \log n)$  – every level does equal work.
- **Case 3** ( $d > \log_b a$ , i.e.  $a < b^d$ ):  $T(n) = \Theta(n^d)$  – the root dominates.

- A7.** (a)  $a = 2, b = 2, d = 1, \log_2 2 = 1 = d \Rightarrow$  Case 2,  $\Theta(n \log n)$ . (b)  $a = 1, b = 2, d = 0, \log_2 1 = 0 = d \Rightarrow$  Case 2,  $\Theta(\log n)$ . (c)  $a = 3, b = 2, d = 1, \log_2 3 \approx 1.585 > 1 \Rightarrow$  Case 1,  $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$ . (d)  $a = 4, b = 2, d = 1, \log_2 4 = 2 > 1 \Rightarrow$  Case 1,  $\Theta(n^2)$ . (e)  $a = 2, b = 2, d = 2, \log_2 2 = 1 < 2 \Rightarrow$  Case 3,  $\Theta(n^2)$ .
- A8.** (a)  $a = 9, b = 3, d = 1, \log_3 9 = 2 > 1 \Rightarrow$  Case 1,  $\Theta(n^2)$ . (b)  $a = 1, b = 2, d = 1, \log_2 1 = 0 < 1 \Rightarrow$  Case 3,  $\Theta(n)$ . (c)  $a = 8, b = 2, d = 3, \log_2 8 = 3 = d \Rightarrow$  Case 2,  $\Theta(n^3 \log n)$ . (d)  $a = 7, b = 2, d = 2, \log_2 7 \approx 2.807 > 2 \Rightarrow$  Case 1,  $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$ .
- A9.** An **inversion** of an array  $A$  is a pair of indices  $(i, j)$  with  $i < j$  but  $A[i] > A[j]$ . A sorted (non-decreasing) array has **0** inversions (no pair is out of order). A strictly reverse-sorted array of length  $n$  has  $\binom{n}{2} = \frac{n(n-1)}{2}$  inversions: *every* one of the  $\binom{n}{2}$  pairs  $i < j$  has  $A[i] > A[j]$ , which is the maximum possible.
- A10.** Run mergesort; whenever MERGE emits an element  $R[j]$  from the right half while  $L$  still has  $|L| - i$  elements remaining (i.e.  $R[j] < L[i]$ , so the algorithm takes from the right), those  $|L| - i$  remaining left elements are each *to the left* of  $R[j]$  in the original array yet *larger* than  $R[j]$  – so each forms exactly one inversion with  $R[j]$ . Thus a single such step accounts for exactly  $|L| - i$  inversions. Summed across all merges, this counts every cross-inversion exactly once; combined with the inversions counted recursively inside each half, the total is the array's inversion count.
- A11.**  $T(n) = 2T(n/2) + \Theta(n)$ , identical to mergesort's, because the only addition to MERGE is incrementing a counter by  $|L| - i$  at each right-emission step –  $O(1)$  extra work per merge step, leaving the per-merge cost  $\Theta(n)$  unchanged. By the Master Theorem (Case 2),  $T(n) = \Theta(n \log n)$ .
- A12.** Binary search:  $T(n) = T(n/2) + \Theta(1)$ . By the Master Theorem with  $a = 1, b = 2, d = 0$  ( $\log_2 1 = 0 = d$ , Case 2),  $T(n) = \Theta(\log n)$ . It is "divide and conquer with  $a = 1$ " because, although the input is split into two halves, the algorithm *recurses into only one* of them – so the recursion tree is a single *path* of length  $\log_2 n$  (not a branching tree as in mergesort), and the total work is just  $\Theta(1)$  per level times  $\log_2 n$  levels.
- A13.** Split  $A$  at the midpoint  $m$ . The maximum-sum subarray is one of: (i) entirely in the left half, (ii) entirely in the right half, or (iii) *crossing* the midpoint. Cases (i) and (ii) are solved by recursion. For (iii), the best crossing subarray is the best suffix of the left half (computed by scanning leftward from  $m$  keeping a running maximum) joined to the best prefix of the right half (scanning rightward) –  $O(n)$  total. Return the max of the three. The recurrence is  $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$  (Master Theorem, Case 2).
- A14.** Exponentiation by squaring uses

$$x^n = \begin{cases} 1 & n = 0, \\ (x^{n/2})^2 & n \text{ even}, \\ x \cdot (x^{(n-1)/2})^2 & n \text{ odd}. \end{cases}$$

Letting  $M(n)$  be the number of multiplications,  $M(n) = M(\lfloor n/2 \rfloor) + O(1)$  (one or two multiplications to square and possibly multiply by  $x$ ). By the Master Theorem ( $a = 1, b = 2, d = 0$ , Case 2),  $M(n) = \Theta(\log n)$ . The key is that  $x^{n/2}$  is computed *once* and squared; if instead we wrote  $x^n = x^{n/2} \cdot x^{n/2}$  and computed each factor separately, the recurrence would become  $M(n) = 2M(n/2) + O(1) = \Theta(n)$  – no savings over naive repeated multiplication.

- A15. Quickselect:** choose a pivot  $p$ , partition  $A$  into elements  $< p$ ,  $= p$ , and  $> p$ ; if the  $k$ -th smallest lies in the  $< p$  part, recurse there; if it is among the  $= p$  part, return  $p$ ; otherwise recurse on the  $> p$  part with  $k$  adjusted by the number of discarded smaller elements. With a *random* pivot, with constant probability the pivot discards a constant fraction of the array, so the expected work satisfies  $T(n) \leq T(cn) + O(n)$  for some constant  $c < 1$  (e.g.  $c = 3/4$ ), a decreasing geometric series summing to  $O(n)$ . The Master Theorem does not apply because (a) only *one* subproblem is solved and (b) its size is a constant *fraction* of  $n$  rather than a fixed  $n/b$  for both branches. The worst case is  $\Theta(n^2)$ , occurring when the pivot is repeatedly the largest or smallest remaining element (e.g. a sorted input with a last-element pivot), so each call shrinks the problem by only one.
- A16. Majority element via D&C:** split into halves; recursively find each half's majority candidate. A value that is a majority of the *whole* (occurs  $> n/2$  times) must be a majority of at least one half – because if it were a strict minority in *both* halves (occurring  $\leq$  half the time in each), it would occur  $\leq n/2$  times overall, a contradiction. So the only candidates are the two halves' majority candidates; the combine step counts each candidate's occurrences across the whole range (an  $O(n)$  scan) and returns whichever exceeds  $n/2$ . Recurrence:  $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$ .
- A17. Karatsuba:**  $a = 3, b = 2, d = 1$ ;  $\log_2 3 \approx 1.585 > 1 \Rightarrow$  Case 1,  $T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$ . Naive D&C:  $a = 4, b = 2, d = 1$ ;  $\log_2 4 = 2 > 1 \Rightarrow$  Case 1,  $T(n) = \Theta(n^2)$ . Both are in Case 1, so the running time is  $\Theta(n^{\log_b a})$ , governed *entirely* by  $a$  (the number of recursive multiplications): dropping  $a$  from 4 to 3 drops the exponent from 2 to  $\log_2 3 \approx 1.585$ . Shaving the  $\Theta(n)$  combine term would not help, because in Case 1 the combine term is asymptotically dominated by the leaf work  $n^{\log_b a}$  – the running time does not depend on  $d$  at all once  $d < \log_b a$ .

## Part B

- B1. (b)**  $T(n) = 2T(n/2) + \Theta(n)$ : two recursive sorts of half-size halves, plus a linear merge.
- B2. True.** Each comparison emits one element, so the total number of comparisons/appends is at most the total length  $n$ ; merging is  $\Theta(n)$ .
- B3. (b)**  $\Theta(n \log n)$ . ( $a = b = 2, d = 1, \log_2 2 = 1 = d$ , Master Theorem Case 2.)
- B4. (c)**  $d < \log_b a$  (equivalently  $a > b^d$ ): leaves dominate,  $T(n) = \Theta(n^{\log_b a})$ .
- B5. 3 inversions:**  $(0, 1)$  since  $3 > 2$ ,  $(0, 2)$  since  $3 > 1$ , and  $(1, 2)$  since  $2 > 1$ . This is the maximum  $\binom{3}{2} = 3$  for a length-3 array.
- B6. False.** Counting inversions can be done in  $\Theta(n \log n)$  via the modified mergesort; the  $\Theta(n^2)$  brute force is not optimal.
- B7. (b)**  $\Theta(\log n)$  – each step halves the exponent.
- B8. (b)**  $\Theta(n)$  expected (with a random pivot). The worst case is  $\Theta(n^2)$ .
- B9. True.** The D&C version is  $\Theta(n \log n)$  (recurrence  $2T(n/2) + \Theta(n)$ ); Kadane's single-pass dynamic program is  $\Theta(n)$ .
- B10.**  $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$ .